2 Struktur Data & Algoritma Dasar



2

Struktur Data & Algoritma Dasar

Pernah Mengalami Ini?

Mari kita mulai pembelajaran ini dari aktivitas yang sering kamu temui sehari-hari. Bayangkan kamu sedang antre membeli makanan di kantin sekolah. Siapa yang datang lebih awal dan berdiri di paling depan, pasti akan dilayani lebih dulu, bukan?

Tapi, coba perhatikan saat petugas kantin menumpuk piring kotor. Piring yang ditaruh paling terakhir (paling atas) justru akan diambil paling pertama saat petugas mulai mencucinya.Dua kejadian ini adalah contoh nyata bagaimana kita mengatur sebuah barang.

Dalam dunia komputer, cara mengatur "antrean" dan "tumpukan" data ini disebut dengan Struktur Data. Setelah data tersimpan dengan rapi, barulah komputer menggunakan Algoritma Dasar untuk memprosesnya, seperti mengurutkan atau mencarinya.

Apa Itu Struktur Data?

Secara sederhana, Struktur Data adalah cara komputer menyimpan, menyusun, dan mengatur sekumpulan data atau objek di dalam memori sehingga data tersebut dapat dikelola, diambil, atau dikeluarkan kembali sewaktu-waktu secara efisien.

Agar lebih mudah dipahami, mari gunakan analogi lemari pakaian.Jika kamu melempar semua bajumu secara acak ke dalam lemari, baju-baju itu memang "tersimpan", tetapi kamu akan kesulitan dan membuang banyak waktu saat mencari seragam sekolah di pagi hari. Namun, jika kamu menggunakan struktur yang jelas (misalnya rak atas khusus untuk seragam, laci bawah untuk kaus kaki, dan gantungan untuk jaket), kamu bisa menemukan pakaian yang kamu butuhkan dalam hitungan detik.

Sama halnya dengan komputer. Agar program tidak lambat atau bingung saat memproses ribuan data, komputer wajib menggunakan struktur penyimpanan data yang tepat. Di dalam dasar ilmu Informatika, ada dua jenis struktur data yang paling sering digunakan untuk mengatur urutan penyimpanan Antrean (Queue) dan Tumpukan (Stack)

1 Antrean (Queue)

Dalam ilmu komputer, Antrean (Queue, dibaca: kyu) adalah sebuah struktur data linear yang berfungsi untuk menyimpan dan mengelola sekumpulan elemen dengan mengusung konsep utama FIFO (First In First Out). Melalui konsep "masuk pertama, keluar pertama" ini, sistem akan memastikan bahwa data yang lebih dulu datang akan selalu diproses lebih awal. Secara teknis, Queue beroperasi layaknya sebuah lorong atau pipa terbuka yang memiliki dua jalur akses terpisah di ujung yang berbeda, yaitu pintu belakang (Rear/Tail) sebagai titik masuk dan pintu depan (Front/Head) sebagai titik keluar.

Oleh karena itu, pergerakan data pada struktur ini mengalir satu arah dari belakang ke depan dengan dua instruksi utama:

- **Enqueue:** Proses memasukkan atau menambahkan data baru ke dalam sistem. Sesuai konsep FIFO, data ini wajib dimasukkan melalui pintu belakang dan ditempatkan di barisan paling akhir antrean. (Catatan: Pada beberapa simulator dasar, instruksi ini sering disamakan dengan perintah Push).
- **Dequeue:** Proses mengambil, memproses, dan mengeluarkan data dari sistem. Operasi ini hanya bisa dilakukan secara eksklusif pada data yang saat ini berada di posisi terdepan. (Pada aplikasi simulator, perintah ini sering disebut Pop Queue).



Prinsip Utama Antrean (Queue)

First In First Out (FIFO).

Contoh dalam Kehidupan Sehari-hari

- Antrean di Kantin: Siswa yang datang lebih dulu dan berdiri di depan kasir akan dilayani dan mendapatkan makanannya paling pertama.
- Gerbang Tol atau Lampu Merah: Mobil yang pertama kali masuk ke barisan antrean dan berhenti di paling depan pasti akan menjadi mobil yang melaju pertama kali.
- Antrean Panggilan Customer Service: Saat kamu menelepon pusat bantuan dan semua petugas sedang sibuk, teleponmu akan dimasukkan ke dalam antrean. Penelepon yang masuk paling awal akan dijawab terlebih dahulu.

2 Tumpukan (Stack)

Tumpukan (Stack) adalah struktur data linear yang menyimpan dan mengelola elemen-elemen komputasi dengan cara disusun secara vertikal atau bertumpuk ke atas berdasarkan konsep LIFO (Last In First Out). Karena mengusung konsep "masuk terakhir, keluar pertama", Stack beroperasi secara mekanis layaknya sebuah tabung silinder yang bagian dasarnya tertutup rapat. Wadah ini hanya memiliki satu pintu akses tunggal di bagian paling atas yang disebut sebagai Puncak (Top), sehingga jalur masuk dan keluarnya berada di titik yang sama.

Karena wadah ini hanya memiliki satu pintu akses, maka mekanisme pergerakan datanya menjadi sangat spesifik melalui dua instruksi berikut:

- **Push:** Proses meng-input atau menyisipkan data baru ke dalam sistem. Setiap data yang masuk akan selalu diletakkan menumpuk di posisi paling atas (puncak tumpukan).
- **Pop (Pop Stack):** Proses mengambil atau menghilangkan data dari sistem. Sesuai konsep LIFO, operasi pelepasan ini hanya bisa dilakukan pada elemen yang berada di posisi teratas. Akibatnya, data yang berada di dasar tumpukan akan terkunci dan tidak bisa diakses sampai seluruh data yang menumpuk di atasnya dibongkar terlebih dahulu.



Prinsip Utama Tumpukan (Stack)

Last In First Out (LIFO).

Contoh dalam Kehidupan Sehari-hari

- Tumpukan Piring Bersih di Prasmanan: Piring yang baru saja dicuci dan diletakkan di tumpukan paling atas adalah piring yang pertama kali akan diambil oleh tamu.
- Memasukkan Pakaian ke dalam Koper: Saat kamu menumpuk pakaian ke dalam koper yang dalam, baju terakhir yang kamu letakkan di posisi teratas akan menjadi baju pertama yang kamu ambil saat membuka koper.
- Menumpuk Kursi Plastik: Setelah acara selesai, kursi-kursi plastik biasanya disusun bertumpuk ke atas. Saat akan digunakan lagi, kursi terakhir yang ditumpuk paling atas akan diambil paling pertama.

Perbandingan Antrean (Queue) vs Tumpukan (Stack)

Saat baru belajar struktur data, orang sering tertukar antara Stack dan Queue karena bingung dengan arah masuk-keluarnya data. Cara termudah membedakannya adalah melalui analogi "pintu". Queue (Antrean) ibarat lorong dengan dua pintu searah: data masuk dari belakang (Enqueue) dan keluar dari depan (Dequeue), sehingga sistemnya adil karena yang datang awal diproses duluan (FIFO).

Sebaliknya, Stack (Tumpukan) ibarat stoples dengan satu pintu di atas, sehingga data yang baru masuk (Push) justru akan terambil lebih dulu (Pop) karena berada di puncak tumpukan (LIFO). Sebagai trik singkat: jika kasusnya membahas barisan orang, mobil, atau layanan, itu pasti Queue. Namun, jika membahas benda yang disusun ke atas seperti buku, piring, atau kardus, gunakan logika Stack.

Tabel Perbandingan

Kriteria Pembanding	Antrean (Queue)	Tumpukan (Stack)
Prinsip Utama	FIFO (First In First Out)	LIFO (Last In First Out)
Logika Pengambilan Data	Data yang pertama kali masuk, akan menjadi yang pertama keluar.	Data yang paling akhir masuk, akan menjadi yang pertama keluar
Analogi Posisi Data	Diambil dari barisan paling depan.	Diambil dari tumpukan paling atas atau barisan paling akhir.
Istilah Memasukkan Data	Enqueue / Push Queue	Push
Istilah Mengeluarkan Data	Dequeue / Pop Queue	Pop / Pop Stack
Contoh Sehari-hari	Mobil saat memasuki gerbang tol	Menyimpan barang besar/box di gudang atau tumpukan piring di hajatan.
Contoh di Komputer	Sistem antrean mencetak dokumen pada mesin printer	Sistem riwayat (History) mundur atau tombol Back pada Web Browser.



Contoh Soal Simulasi

Andi sedang bermain menggunakan alat simulator data di komputer. Simulator ini bisa menjalankan operasi tumpukan dan antrean sekaligus pada satu wadah yang sama. Andi memberikan instruksi kepada simulator secara berurutan sebagai berikut:

Langkah	Operasi	Langkah	Operasi
1	Push 3	5	Push 5
2	Push 1	6	Pop Stack
3	Push 9	7	Push 6
4	Pop Queue	8	Pop Queue

- **Pertanyaan A:**
  Angka berapakah yang terhapus (dikeluarkan) tepat pada instruksi langkah ke-6?
- **Pertanyaan B:**
  Angka berapakah yang tersisa di dalam simulator saat seluruh langkah selesai dijalankan?



Pembahasan

Pembahasan (Langkah demi Langkah), Mari kita catat pergerakan datanya. Bayangkan kita memasukkan angka ke dalam wadah dari arah kiri (depan) ke kanan (belakang/atas).

1. **Langkah 1** (Push 3): Masukkan angka 3.
   - Isi Wadah: [3]
2. **Langkah 2** (Push 1): Masukkan angka 1 ke barisan akhir.
   - Isi Wadah: [3, 1]
3. **Langkah 3** (Push 9): Masukkan angka 9 ke barisan akhir.
   - Isi Wadah: [3, 1, 9]
4. **Langkah 4** (Pop Queue): Instruksi Queue menggunakan prinsip FIFO, maka hilangkan angka pada barisan paling depan (yang masuk paling awal). Angka terdepan adalah 3.
   - Isi Wadah Sekarang: [1, 9]

Lanjutan Pembahasan

5. **Langkah 5** (Push 5): Masukkan angka 5 ke barisan akhir.
   - Isi Wadah: [1, 9, 5]
6. **Langkah 6** (Pop Stack): Instruksi Stack menggunakan prinsip LIFO, maka hilangkan angka pada barisan paling akhir/teratas (yang baru saja masuk). Angka terakhir adalah 5.
   - Isi Wadah Sekarang: [1, 9]
7. **Langkah 7** (Push 6): Masukkan angka 6 ke barisan akhir.
   - Isi Wadah: [1, 9, 6]
8. **Langkah 8** (Pop Queue): Kembali ke prinsip FIFO, hilangkan angka yang saat ini berada di barisan paling depan. Angka terdepan saat ini adalah 1.
   5. • Isi Wadah Terakhir: [9, 6]



Jawaban

- **Jawaban Pertanyaan A**: Angka yang dihilangkan pada langkah ke-6 adalah angka 5.
- **Jawaban Pertanyaan B**: Angka yang tersisa dan bertahan di dalam simulator hingga akhir adalah angka 9 dan 6.

Apa Itu Algoritma Dasar

Jika Struktur Data kita ibaratkan sebagai "wadah" atau rak penyimpanan, maka Algoritma Dasar adalah instruksi atau panduan langkah demi langkah untuk mengelola isi dari wadah tersebut. Komputer tidak akan tahu apa yang harus dilakukan terhadap data yang disimpannya jika tidak diberi perintah yang logis dan sistematis. Algoritma inilah yang menjadi "otak penggerak" agar data bisa diproses.

Di dalam sistem dasar komputasi, jumlah data yang masuk sering kali masif dan acak. Oleh karena itu, terdapat dua pekerjaan paling utama dan paling sering diinstruksikan kepada komputer agar data tersebut mudah dikelola, yaitu:

- Menemukan sebuah informasi spesifik di tengah tumpukan data dengan cepat dan akurat (Pencarian / Searching).
- Menyusun dan merapikan sekumpulan data yang posisinya masih acak agar tersusun mengikuti pola atau aturan tertentu (Pengurutan / Sorting).

Algoritma Pencarian (Searching)

Pencarian (Searching) adalah sebuah proses komputasi sistematis berupa langkah demi langkah untuk menemukan letak atau keberadaan suatu data tertentu di dalam kumpulan data yang besar. Bayangkan jika sebuah aplikasi tidak memiliki algoritma pencarian; kita harus menggulir daftar kontak satu per satu hanya untuk mencari satu nomor telepon.

Dalam dasar ilmu komputer, terdapat dua jenis metode pencarian utama yang bekerja dengan cara yang sangat berbeda, yaitu Pencarian Berurutan (Sequential Search) dan Pencarian Biner (Binary Search).

1 Pencarian Berurutan (Sequential Search/Linear Search)

Pencarian berurutan, atau yang sering juga disebut sebagai Linear Search, adalah algoritma pencarian yang paling dasar dan sederhana. Sesuai dengan namanya, cara kerja algoritma ini adalah dengan menelusuri data satu per satu, mulai dari elemen paling awal (paling ujung kiri), kemudian bergeser satu langkah ke kanan, dan seterusnya hingga data yang dicari berhasil ditemukan atau hingga mencapai akhir dari kumpulan data.

Keunggulan:

Metode ini sangat tangguh dan bisa digunakan pada data yang kondisinya masih acak atau belum diurutkan. Komputer tidak peduli apakah data tersebut berantakan; ia hanya akan membacanya satu per satu secara linear.

Kelemahan:

Jika data yang dicari kebetulan berada di posisi paling akhir (atau bahkan tidak ada sama sekali di dalam kumpulan data), komputer akan membuang waktu komputasi yang sangat lama karena harus mengecek ratusan atau ribuan data sebelumnya secara sia-sia.

Contoh dalam Kehidupan Sehari-hari:

Mengecek daftar hadir siswa yang baru saja dikumpulkan dan belum disusun berdasarkan urutan abjad. Jika guru ingin mencari siswa bernama "Zidan", guru tersebut terpaksa harus membaca setiap nama satu per satu dari halaman pertama dari atas ke bawah hingga nama tersebut ditemukan.



Kata Kunci Pencarian Berurutan (Sequential Search/Linear Search)

Mencari satu per satu dari awal, Cocok untuk data acak.

2 Pencarian Biner (Binary Search)

Pencarian Biner adalah algoritma pencarian yang dirancang untuk bekerja sangat cepat dan efisien dalam memproses data berskala besar. Alih-alih mengecek data secara linier dari depan, algoritma ini langsung "menebak" ke tengah-tengah kumpulan data.

Cara kerjanya adalah Komputer akan melihat elemen tengah. Jika elemen tengah tersebut bukanlah data yang dicari, komputer akan mengevaluasi apakah data yang dicari nilainya lebih kecil atau lebih besar dari elemen tengah tersebut. Berdasarkan evaluasi tersebut, komputer akan membuang tepat separuh kumpulan data yang sudah pasti salah, dan mengulangi proses pencarian nilai tengah pada separuh data yang tersisa.

Syarat Mutlak (Wajib):

Algoritma ini TIDAK BISA dijalankan pada data yang acak. Agar komputer bisa membuang separuh data ke kiri atau ke kanan dengan tepat, himpunan datanya wajib sudah dalam keadaan terurut terlebih dahulu (misal: sudah diurutkan dari nilai terkecil ke terbesar).

Keunggulan:

Kecepatannya sangat luar biasa. Menemukan satu data dari dalam jutaan data acak dengan algoritma ini hanya membutuhkan beberapa kali perbandingan saja, berbeda dengan Sequential Search/Linear search yang mungkin membutuhkan jutaan perbandingan.

Contoh dalam Kehidupan Sehari-hari:

Mencari kata "Sekolah" di dalam sebuah kamus bahasa Indonesia setebal 1.000 halaman (ingat, isi kamus sudah pasti terurut abjadnya). Kamu tidak akan mungkin membaca isi kamus dari halaman pertama (huruf A) satu per satu. Kamu akan langsung membelah dan membuka bagian tengah kamus (misalnya terbuka di huruf M). Karena kamu tahu bahwa huruf S posisinya pasti ada setelah huruf M, kamu langsung membuang atau mengabaikan seluruh halaman di sebelah kiri, dan memfokuskan pencarian di belahan buku sebelah kanan. Proses membelah tengah ini terus diulang hingga kata "Sekolah" ditemukan.



Kata Kunci Pencarian Biner (Binary Search)

Membelah nilai tengah, Syarat: Data WAJIB terurut.



Contoh Soal Logika Pencarian

- **Soal 1 (Simulasi Sequential Search)**
  Sebuah sistem komputer menyimpan deret angka acak di dalam memori sebagai berikut: 8, 3, 10, 5, 2Jika program diperintahkan untuk mencari keberadaan angka 5 menggunakan metode Sequential Search, berapa kali program tersebut harus melakukan perbandingan sebelum data ditemukan?
- **Soal 2 (Simulasi Binary Search)**
  Diberikan sebuah deret angka yang sudah dalam kondisi terurut: 2, 4, 6, 8, 10, 12, 14Sebuah program diperintahkan mencari posisi angka 12 menggunakan metode Binary Search. Jelaskan langkah-langkah logika pembandingan yang dilakukan program tersebut!



Pembahasan

Soal 1 (Simulasi Sequential Search/Linear Search)

Komputer akan mengecek indeks posisi dari sebelah kiri secara linier:

1. Pengecekan Pertama: Apakah elemen urutan pertama (8) bernilai 5? (Salah).
2. Pengecekan Kedua: Apakah elemen urutan kedua (3) bernilai 5? (Salah).
3. Pengecekan Ketiga: Apakah elemen urutan ketiga (10) bernilai 5? (Salah).
4. Pengecekan Keempat: Apakah elemen urutan keempat (5) bernilai 5? (Benar! Data Ditemukan).

Jawaban: Program membutuhkan tepat 4 kali pengecekan/perbandingan untuk menemukan angka tersebut.

Soal 2 (Simulasi Binary Search)

Karena menggunakan metode pencarian biner, komputer akan berfokus mencari dan membandingkan nilai tengah pada setiap iterasinya.

1. Iterasi Pertama:

   Komputer melihat keseluruhan baris: [2, 4, 6, 8, 10, 12, 14].Elemen tengahnya adalah angka 8. Komputer membandingkan: Apakah 12 sama dengan 8? (Tidak). Karena 12 lebih besar dari 8, komputer yakin bahwa target pasti berada di sisi kanan. Angka 8 beserta seluruh sisi kirinya (2, 4, 6) dieliminasi.

Lanjutan Pembahasan

2. Iterasi Kedua:

Komputer fokus pada sisa baris yang tersisa: [10, 12, 14].Elemen tengah dari baris baru ini adalah angka 12.Komputer membandingkan: Apakah 12 sama dengan 12? (Benar!). Data telah ditemukan.

Jawaban: Program berhasil menemukan target hanya dalam 2 langkah iterasi perbandingan karena berhasil memotong separuh dari himpunan data pada setiap langkahnya.

Algoritma Pengurutan (Sorting)

Dalam dunia komputasi, pengurutan (Sorting) adalah proses mengatur sekumpulan data yang acak agar tersusun secara teratur berdasarkan aturan tertentu. Pengurutan ini sangat penting karena data yang sudah terurut akan jauh lebih cepat dan mudah saat dicari (Searching).Ada dua arah pengurutan yang standar digunakan oleh komputer:

- **Ascending (Naik):** Mengurutkan data dari nilai paling kecil ke nilai paling besar (Contoh: A ke Z, atau angka 1 ke 10).
- **Descending (Turun):** Mengurutkan data dari nilai paling besar ke nilai paling kecil (Contoh: Z ke A, atau angka 10 ke 1).

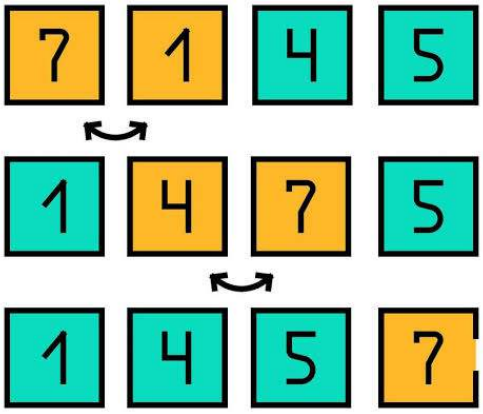
Untuk melakukan tugas ini, komputer menggunakan beberapa metode instruksi (Algoritma). Tiga metode dasar yang paling sering dipelajari adalah:

1 Pengurutan Gelembung (Bubble Sort)

Metode ini merupakan algoritma pengurutan yang paling dasar dan paling pertama diajarkan dalam ilmu komputer karena logikanya sangat sederhana. Konsep utama dari Bubble Sort tidak berfokus pada memindai keseluruhan data sekaligus, melainkan berfokus penuh pada pembandingan dua elemen data yang letaknya saling bersebelahan. Komputer akan terus-menerus mengecek pasangan data secara berurutan, memastikan bahwa elemen di sebelah kiri tidak boleh lebih besar dari elemen di sebelah kanan (jika urutannya ascending). Jika posisinya salah, komputer akan langsung menukarnya. Nama "Bubble" (Gelembung) diambil dari fenomena alam di mana gelembung udara yang ringan atau benda yang nilainya paling besar akan perlahan-lahan terdorong dan naik ke permukaan (ke ujung akhir barisan) melalui proses pertukaran yang berulang-ulang ini.

Cara Kerjanya (Analogi Barisan):

Sebagai guru, kamu tidak melihat seluruh barisan sekaligus. Matamu hanya fokus melihat pada siswa nomor 1 dan siswa nomor 2. Kamu membandingkan keduanya: Apakah siswa nomor 1 lebih tinggi dari nomor 2? Jika iya, posisi mereka salah, sehingga kamu menyuruh mereka bertukar tempat. Jika tidak, biarkan saja. Selanjutnya, kamu bergeser fokus melihat siswa nomor 2 dan siswa nomor 3, membandingkan kembali, dan menyuruh mereka bertukar jika nomor 2 lebih tinggi. Proses bergeser dan menukar ini kamu lakukan langkah demi langkah, tanpa henti, dari ujung kiri hingga sampai ke ujung kanan barisan.Hasil



Akhir Iterasi:

Akibat dari proses pertukaran beruntun ini, siswa yang posturnya paling tinggi akan terus-menerus didorong dan digeser ke arah belakang, hingga pada akhir putaran pertama, siswa tertinggi tersebut berhasil mencapai urutan paling akhir (menggelembung ke permukaan). Komputer akan mengulangi putaran ini dari awal berkali-kali hingga tidak ada lagi posisi yang perlu ditukar.

Kelebihan:

Sangat mudah dipahami secara logika dan mudah dikodekan (diprogram) oleh programmer pemula. Cocok untuk menyortir himpunan data dalam jumlah yang sangat kecil.

Kekurangan:

Sangat lambat dan memakan memori jika memproses data berjumlah besar, karena komputer harus membuang terlalu banyak waktu untuk melakukan aksi pertukaran (swapping) secara berulang-ulang di setiap langkahnya.



Kata Kunci Pengurutan Gelembung (Bubble Sort)

Bandingkan dengan Sebelah, Tukar jika salah posisi, Angka terbesar bergeser (menggelembung) ke akhir barisan.

Contoh Soal Logika Pengurutan (Bubble Sort)



Soal

Sebuah program komputer menggunakan algoritma Bubble Sort untuk mengurutkan deret angka acak berikut dari nilai yang terkecil ke yang terbesar (Ascending).Susunan data awal di dalam memori: 7, 3, 9, 5



Pembahasan

Ingat hukum Bubble Sort: Komputer selalu membandingkan 2 angka yang letaknya bersebelahan, mulai dari arah kiri. Jika angka di sebelah kiri ternyata lebih besar, maka posisinya wajib ditukar.

- Susunan Awal: 7, 3, 9, 5
- Langkah 1 (Cek posisi 1 & 2): Bandingkan angka 7 dan 3. Karena 7 lebih besar dari 3, posisinya ditukar.
  [Susunan berubah menjadi: 3, 7, 9, 5]
- Langkah 2 (Cek posisi 2 & 3): Bandingkan angka 7 dan 9. Karena 7 tidak lebih besar dari 9, posisinya tetap (tidak ditukar).
  Susunan tetap: 3, 7, 9, 5]
- Langkah 3 (Cek posisi 3 & 4): Bandingkan angka 9 dan 5. Karena 9 lebih besar dari 5, posisinya ditukar.
  Susunan berubah menjadi: 3, 7, 5, 9]

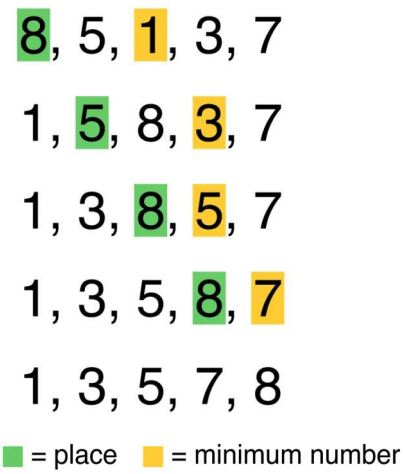


Jawaban

Iterasi putaran pertama selesai. Susunan deret pada akhir putaran pertama adalah 3, 7, 5, 9.(Penting: Perhatikan bahwa tujuan Bubble Sort terbukti. Angka paling besar yaitu 9 sudah sukses bergeser/menggelembung dan diamankan ke posisi paling akhir).

2 Pengurutan Seleksi (Selection Sort)

Metode ini lahir sebagai jawaban untuk menutupi kelemahan utama dari Bubble Sort, yaitu terlalu banyaknya proses tukar-menukar (swapping) tempat yang membuat mesin komputer cepat kelelahan. Konsep utama dari Selection Sort berfokus penuh pada pemindaian (scanning) keseluruhan data untuk menemukan nilai yang paling ekstrem (paling kecil atau paling besar, tergantung arah pengurutan).



Alih-alih menukar data yang bersebelahan berkali-kali, komputer justru akan menyisir seluruh barisan data acak dari awal hingga akhir hanya untuk mencari satu angka yang nilainya paling kecil. Setelah target tersebut ditemukan, komputer tidak akan menggesernya sedikit demi sedikit. Komputer akan langsung menarik angka terkecil tersebut dan menukarnya sekaligus dengan angka yang sedang berada di posisi terdepan. Setelah itu, urutan terdepan tersebut akan "dikunci" secara permanen. Nama "Selection" (Seleksi) diambil dari sifat algoritma ini yang menyeleksi (memilih) kandidat terbaik dari seluruh tumpukan data untuk langsung dipindahkan ke posisi yang seharusnya.

Cara Kerjanya (Analogi Barisan):

Berbeda dengan metode sebelumnya, sebagai guru, kamu tidak lagi menyuruh anak-anak saling bertukar posisi secara berantai. Alih-alih demikian, matamu akan langsung memindai (menyeleksi) seluruh barisan anak yang acak tersebut dari ujung ke ujung. Tujuan pemindaianmu hanya satu: Mencari dan menandai siapa anak yang posturnya paling pendek di antara semuanya. Setelah menemukannya, kamu menarik anak tersebut keluar, lalu menukarnya posisi berdirinya dengan anak yang sedang berdiri di urutan paling depan (nomor 1). Setelah posisi nomor 1 terkunci secara permanen, kamu mengulangi pemindaian pada sisa anak yang belum terurut untuk mencari siapa yang terpendek di antara mereka, dan menempatkannya di posisi nomor 2.

Akhir Iterasi:

Pada akhir setiap putaran iterasi, nilai terkecil dari himpunan data acak yang tersisa akan berhasil ditemukan dan langsung dikunci letaknya di urutan barisan terdepan secara berangsur-angsur.

Kelebihan:

Karena data langsung dipindahkan ke posisi yang tepat tanpa harus saling menggeser sedikit demi sedikit, algoritma ini memangkas jumlah aksi pertukaran (swapping) secara drastis, sehingga beban memori kerja komputer menjadi lebih ringan dibandingkan Bubble Sort.

Kekurangan:

Pada akhir setiap putaran iterasi, nilai terkecil dari himpunan data acak yang tersisa akan berhasil ditemukan dan langsung dikunci letaknya di urutan barisan terdepan secara berangsur-angsur.



Pengurutan Seleksi (Selection Sort)

Cari Nilai Terkecil/Terbesar, Tukar dengan Posisi Terdepan, Posisinya Terkunci Permanen.

Contoh Soal Logika Pengurutan (Selection Sort)



Soal

Sebuah program komputer diperintahkan menggunakan algoritma Selection Sort untuk mengurutkan deret angka acak berikut dari nilai yang terkecil ke yang terbesar (Ascending).Susunan data awal di dalam memori: 8, 4, 9, 2Tuliskan hasil susunan angka tersebut hanya setelah komputer menyelesaikan iterasi (putaran) pertama!



Pembahasan

Ingat hukum Selection Sort: Komputer tidak menukar angka yang bersebelahan. Komputer menyeleksi (scanning) seluruh barisan untuk mencari angka paling kecil, lalu menukarnya secara langsung dengan angka yang berada di urutan terdepan.

- Susunan Awal: 8, 4, 9, 2
- Langkah 1 (Proses Seleksi / Scanning): Komputer melihat seluruh barisan dan mencari nilai terkecil. Kandidat yang ditemukan adalah angka 2.
- Langkah 2 (Proses Tukar / Swap): Komputer menarik angka 2 tersebut, lalu menukarnya langsung dengan angka yang menduduki posisi urutan terdepan (yaitu angka 8).



Jawaban

Iterasi putaran pertama selesai. Posisi angka 2 dan 8 saling bertukar, sehingga susunannya berubah menjadi 2, 4, 9, 8.(Penting: Pada akhir putaran ini, angka terkecil yaitu 2 sudah sukses dikunci secara permanen di posisi nomor 1, sehingga tidak akan diganggu lagi pada putaran berikutnya).

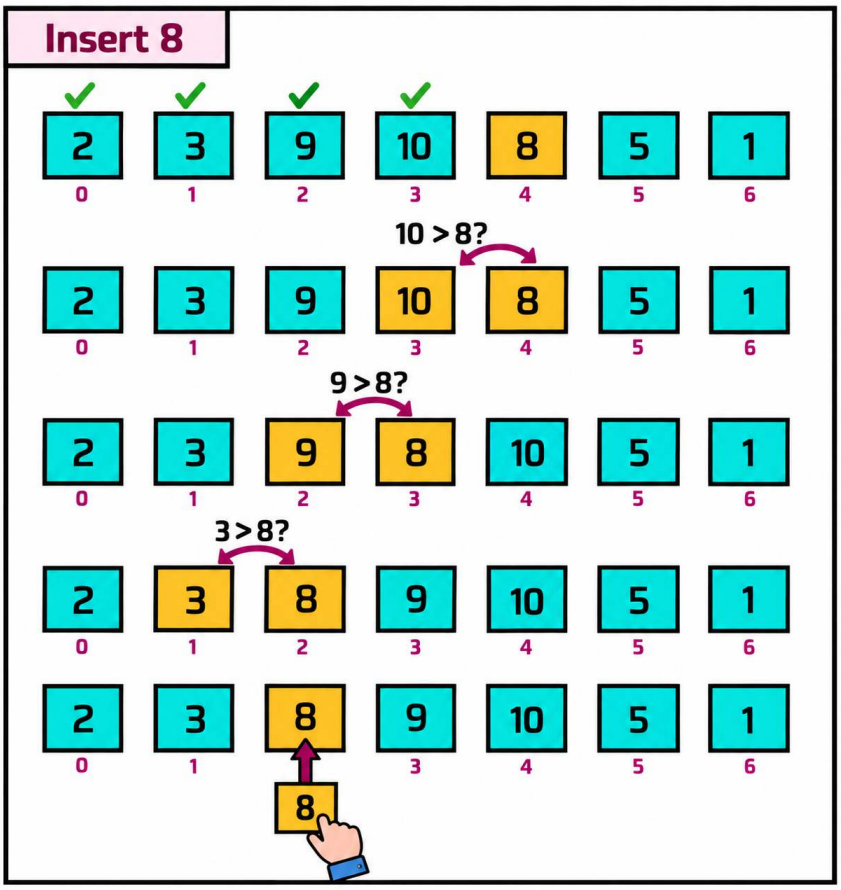
3 Pengurutan Sisip (Insertion Sort)

Metode pengurutan yang ketiga ini menggunakan pendekatan logis yang sangat berbeda dari Bubble Sort maupun Selection Sort. Algoritma ini dirancang dengan meniru persis insting alami otak manusia saat sedang merapikan benda di kehidupan nyata. Konsep utama Insertion Sort adalah mengambil satu demi satu elemen dari himpunan data yang acak, lalu menyisipkannya (menyelipkannya) secara langsung ke posisi koordinat yang benar di antara sekumpulan data lain yang statusnya sudah terurut.

Dalam proses komputasinya, mesin akan membagi memori kerjanya menjadi dua sisi imajiner: sisi "sudah urut" (di sebelah kiri) dan sisi "masih acak" (di sebelah kanan). Komputer akan terus mencomot elemen terdepan dari kubu yang masih acak, lalu membawanya ke kubu yang sudah urut, dan menyelipkannya tepat di tengah-tengah posisi yang sesuai agar sisi yang sudah urut tersebut tetap rapi.

Cara Kerjanya (Analogi Barisan):

Sebagai guru, kamu tidak perlu membandingkan seluruh anak secara membingungkan. Kamu memulai dengan menganggap siswa pertama yang berdiri di urutan terdepan sebagai "barisan yang sudah terurut" (meskipun ia baru sendirian). Kemudian, kamu memanggil siswa urutan kedua. Kamu melihat ukuran tubuhnya, lalu membandingkannya dengan siswa pertama. Kamu langsung menyisipkan (insert) siswa kedua tersebut ke posisi yang benar: apakah ia harus berdiri di depan siswa pertama, atau di belakangnya. Selanjutnya, kamu memanggil siswa urutan ketiga. Kamu membandingkan tingginya dengan dua siswa pertama yang sudah berbaris rapi tadi, lalu menyisipkannya tepat ke tengah-tengah posisi yang sesuai agar ketiganya membentuk barisan terurut. Kamu terus memanggil sisa siswa yang masih acak satu per satu, dan menyisipkan mereka ke posisi yang tepat pada barisan yang sudah rapi tersebut.



Hasil Akhir Iterasi

Komputer memproses satu iterasi dengan cara memindahkan tepat satu elemen dari kumpulan data acak, dan menyisipkannya ke ruang kosong yang digeser paksa di antara deret data yang telah diurutkan sebelumnya, sehingga area data yang sudah urut makin lama makin panjang.

Kelebihan:

Algoritma ini memiliki kinerja yang sangat lincah, cepat, stabil, dan sangat adaptif. Jika data yang diterima sejak awal ternyata kondisinya hampir terurut atau tidak terlalu acak berantakan, algoritma ini akan menyelesaikannya dalam waktu kilat karena proses sisipannya sangat minim.

Kekurangan

Akan menjadi sangat tidak efisien, membuang banyak waktu, dan membebani daya prosesor jika sistem menerima data yang kondisinya benar-benar terbalik 180 derajat (worst-case scenario). Hal ini karena setiap kali komputer akan menyisipkan satu elemen baru, ia dipaksa harus menggeser mundur seluruh letak data lain yang sebelumnya sudah rapi.enyelesaikannya dalam waktu kilat karena proses sisipannya sangat minim.

Contoh Soal Logika Pengurutan (Insertion Sort)



Soal

Sebuah program menggunakan algoritma Insertion Sort untuk mengurutkan deret berikut dari nilai yang terkecil ke yang terbesar (Ascending).Susunan data awal di dalam memori: 5, 8, 3, 9. Jelaskan langkah-langkah logika (pembandingan dan pergeseran) yang terjadi pada saat komputer mengeksekusi Iterasi Kedua, yaitu pada saat komputer mencoba menyisipkan elemen ketiga (angka 3)!



Pembahasan

Ingat hukum Insertion Sort: Komputer membagi barisan menjadi dua bagian imajiner (kiri yang sudah rapi, dan kanan yang masih acak). Komputer mengambil satu angka dari bagian acak, lalu menyelipkannya ke koordinat yang tepat di dalam barisan yang sudah rapi tersebut.

- Susunan Awal: 5, 8, 3, 9
- Kondisi Awal (Hasil Iterasi Pertama): Pada langkah sebelumnya, komputer telah merapikan angka 5 dan 8. Jadi, bagian "sudah rapi" adalah [5, 8], sedangkan bagian "masih acak" adalah [3, 9].
- Iterasi Kedua (Proses Penyisipan Angka 3):
  - Komputer mencomot angka 3 dari bagian yang acak. (Angka 3 dipegang oleh memori).
  - Komputer mengecek barisan yang sudah rapi [5, 8] dari arah kanan (belakang) ke kiri.
  - Bandingkan dengan 8: Apakah 3 lebih kecil dari 8? Ya. Maka angka 8 digeser mundur ke kanan untuk memberi ruang.
  - Bandingkan dengan 5: Apakah 3 lebih kecil dari 5? Ya. Maka angka 5 digeser mundur ke kanan untuk memberi ruang.
  - Karena sudah tidak ada angka lagi untuk dibandingkan (sudah mentok di depan), komputer menyisipkan (insert) angka 3 ke ruang kosong paling depan tersebut.



Jawaban

Pada iterasi ini, angka 5 dan 8 digeser mundur ke kanan, lalu angka 3 disisipkan ke posisi paling depan. Susunan data berubah menjadi 3, 5, 8, 9.



Glosarium

Algoritma:
Serangkaian instruksi atau langkah-langkah komputasi yang terstruktur dan sistematis untuk memecahkan suatu masalah atau menyelesaikan suatu tugas.

Ascending:
Proses mengurutkan sekumpulan data dari nilai yang terkecil menuju nilai yang terbesar (naik).

Binary Search:
Algoritma pencarian efisien yang bekerja dengan cara menebak nilai tengah secara berulang dan membuang separuh data yang salah. Syarat mutlaknya: data wajib sudah terurut.

Bubble Sort:
Algoritma pengurutan dasar yang bekerja dengan membandingkan dan menukar posisi dua elemen data yang bersebelahan secara terus-menerus.

Data Structure (Struktur Data):
Cara komputer mengatur, menyusun, dan menyimpan sekumpulan data di dalam memori agar dapat diakses atau diolah secara efisien.

Dequeue:
Istilah teknis untuk proses mengeluarkan atau menghapus elemen data dari bagian depan sebuah Queue (antrean).

Descending:
Proses mengurutkan sekumpulan data dari nilai yang terbesar menuju nilai yang terkecil (turun).

Enqueue:
Istilah teknis untuk proses memasukkan atau menambahkan elemen data baru ke bagian paling belakang dari sebuah Queue (antrean).



Glosarium

FIFO (First In, First Out):
Prinsip dasar Queue (antrean) di mana data yang pertama kali masuk ke dalam memori akan menjadi data yang pertama kali diproses atau dikeluarkan.

Insertion Sort:
Algoritma pengurutan yang bekerja dengan cara mengambil elemen data satu per satu, lalu menyisipkannya (insert) ke koordinat yang tepat di dalam deret data yang sudah terurut sebelumnya (seperti menyusun kartu).

Iterasi (Iteration):
Proses perulangan langkah atau putaran yang dilakukan oleh komputer dalam menjalankan sebuah algoritma hingga suatu kondisi terpenuhi.

LIFO (Last In, First Out):
Prinsip dasar Stack (tumpukan) di mana data yang paling terakhir dimasukkan akan menjadi data yang pertama kali diproses atau dikeluarkan (karena posisinya ada di puncak tumpukan).

Pop:
Istilah teknis untuk proses mengambil, menghapus, atau mengeluarkan elemen data teratas dari sebuah Stack (tumpukan)

Push:
Istilah teknis untuk proses memasukkan atau menumpuk elemen data baru ke posisi paling atas pada sebuah Stack (tumpukan).

Queue (Antrean):
Model struktur data linier yang memiliki dua ujung terbuka (satu untuk masuk, satu untuk keluar) dan mengadopsi prinsip FIFO.



Glosarium

Searching (Pencarian):
Proses komputasi algoritmik untuk mencari dan menemukan letak suatu elemen spesifik dari dalam sekumpulan data.

Selection Sort:
Algoritma pengurutan yang bekerja dengan cara memindai (scanning) seluruh data untuk menyeleksi nilai terkecil/terbesar, lalu menukarnya langsung ke urutan paling depan secara permanen

Sequential Search (Linear Search):
Algoritma pencarian dasar yang menelusuri atau mengecek elemen data secara berurutan satu per satu dari ujung ke ujung.

Sorting (Pengurutan):
Proses komputasi algoritmik untuk merangkai sekumpulan data acak agar tersusun secara teratur berdasarkan pola tertentu (biasanya ascending atau descending)

Stack (Tumpukan):
Model struktur data linier yang hanya memiliki satu ujung terbuka untuk akses masuk dan keluar data, serta mengadopsi prinsip LIFO

Swap:
Tindakan teknis dalam pemrograman untuk menukar posisi antara dua elemen data (sangat umum digunakan dalam logika pengurutan).